Radar Inference Accelerator — MPW Project Proposal

1. Project Overview

This project designs and tapes out a radar inference accelerator through an MPW shuttle program targeting SkyWater 130nm via Efabless. The chip is a complete radar signal processing pipeline: a matched filter front end processes raw radar returns into feature vectors, which are passed to a 16×16 systolic array for neural network inference, producing a target classification output. The accelerator interfaces with an on-chip RISC-V host processor (PicoRV32) via Wishbone bus, requiring no external FPGA or microcontroller.
The final deliverable is a GDSII layout submitted to an MPW shuttle in Spring 2027.

2. Motivation

Radar signal processing is computationally intensive and latency-critical. Modern applications — autonomous vehicles, drone detection, satellite SAR — require real-time target classification that general-purpose CPUs cannot deliver efficiently. Custom silicon accelerating both the DSP preprocessing and the inference pipeline addresses this directly.
This project is also well-positioned for research output. The combination of a matched filter front end and systolic array inference on a single tapeout is underexplored in open-source silicon, and NYU's active RF and wireless research groups provide a natural path toward collaboration and funding extensions.

3. Target Computation

The chip accelerates a two-stage pipeline:

Stage 1 — Matched Filter Front End:
x_compressed = IFFT(FFT(received_signal) .* conj(FFT(reference_chirp)))
The received radar return is transformed to the frequency domain via FFT, multiplied pointwise against the conjugate of the stored reference chirp spectrum, then transformed back via IFFT. This is pulse compression — the standard range processing step in radar receivers. The output is a range-compressed profile whose peak location identifies target range. This compressed output becomes the feature vector for the inference stage.

Stage 2 — Neural Network Inference:
y = ReLU(W * x_compressed + b)
W is a quantized INT8 weight matrix computed by a 16×16 systolic array. The nonlinear activation unit applies ReLU and requantization. The argmax of the output vector is the target classification.

4. Architecture

4.1 System Overview

The chip integrates five blocks on a single Sky130 die inside the Efabless Caravel harness:

Caravel harness provides PicoRV32 RISC-V host, Wishbone bus, GPIO, clock management — no external processor needed
Matched filter front end preprocesses raw radar samples into a compressed feature vector
Systolic array performs matrix-vector multiply
Nonlinear activation unit applies ReLU and requantization
Wishbone slave interface connects everything to the host

4.2 Block Descriptions

Matched Filter Front End

A pipelined fixed-point matched filter consisting of three stages: a forward FFT, a complex multiply stage, and an inverse FFT. The forward FFT transforms incoming complex radar samples to the frequency domain. The complex multiply stage multiplies each bin pointwise against the conjugate of a stored reference chirp spectrum — this is the matching step. The IFFT transforms back to produce a range-compressed profile. FFT and IFFT share the same butterfly hardware. The complex multiply stage consists of N parallel complex multipliers (each four real multiplies and two adds), one per frequency bin. FFT size parameterized — target 64-point for initial tapeout.

16×16 Systolic Array

The core compute engine. A grid of 256 processing elements (PEs), each containing one INT8 multiplier and one INT32 accumulator. Input features enter from the left edge, weights are preloaded and flow downward, partial sums accumulate rightward. Every PE performs one MAC per cycle — 256 MACs per cycle vs 1 MAC per cycle for a sequential design. Array size is a single compile-time parameter.

Nonlinear Activation Unit

Combinational block. Takes INT32 accumulator outputs from the systolic array, applies requantization scaling (Q16.16 fixed-point multiply + right shift), clips to INT8 range, and applies ReLU. Small area, straightforward verification.

Wishbone Slave Interface

Memory-mapped register file. Host writes weights and input features, asserts a start bit, polls a done flag, then reads output classification. Standard Wishbone B4 protocol, natively compatible with Caravel harness.

Control FSM

Orchestrates the full pipeline: load weights → load radar samples → run matched filter → feed compressed output to systolic array → activate → write results. Clean state machine with well-defined handshake signals between blocks.

4.3 Dataflow

PicoRV32 (Wishbone) → weight SRAM → systolic array
→ radar samples → matched filter front end → systolic array input
systolic array → activation unit → output register → PicoRV32

5. RTL Design Plan
Language: SystemVerilog throughout
Module breakdown and:

- filter front end (FFT + complex multiply + IFFT)
- 16×16 systolic array
- Nonlinear activation unit
- Wishbone slave interface
- Control FSM

Design approach: Each module developed and unit-tested independently with a Python golden model before integration. Top-level integration testbench validates full pipeline end-to-end.

6. RTL Synthesis
Primary tool: Yosys + OpenLane (open-source, Sky130-compatible, free)
Backup: Synopsys Design Compiler (available via NYU CAD tools license)
Standard cell library: sky130_fd_sc_hd (high-density, well-characterized)
Key synthesis tasks:

Define clock constraint — target 50MHz on Sky130 (conservative, achievable)
Identify critical path — likely through the INT8 multiplier chain in the systolic array PE or the butterfly arithmetic in the FFT
Area estimation — 16×16 array with INT8 PEs and 64-point FFT expected well within 10mm² user area
Gate-level netlist generation and formal equivalence check against RTL

7. Physical Design
Tool: OpenROAD (integrated in OpenLane flow) + Magic + KLayout
Steps:

Floorplanning — place systolic array as a regular grid in the center of the user area, matched filter front end and control logic in surrounding regions, SRAM macros at edges
Power planning — VDD/VSS ring + mesh, ensure IR drop within Sky130 limits
Placement — OpenROAD global + detail placement, exploit regularity of systolic array for dense packing
Clock Tree Synthesis — single clock domain, balanced tree to all 256 PE flip-flops and FFT pipeline stages
Routing — OpenROAD global + detailed routing, DRC-clean target on first pass

Area estimate: 16×16 systolic array dominates. Each PE is approximately 50-100 standard cells. 64-point FFT butterfly network adds comparable area. Total well within Sky130 user area limits.

8. Verification

Functional verification:
Each module has an independent unit testbench before integration. Testbenches are written using Cocotb with a Verilator simulation backend — this keeps testbench logic in Python, which means the golden model and the stimulus/checking logic live in the same language with no context switching.

Golden model is a Python/NumPy implementation of the full pipeline:
numpy FFT → complex multiply → IFFT → INT8 matmul → ReLU → argmax
End-to-end test procedure: generate a synthetic radar chirp in Python, add a simulated target at a known range delay, pass through the golden model, pass the same input through the RTL simulation, and verify outputs match to within quantization error.

Physical verification:
DRC → Magic DRC + KLayout DRC scripts
LVS → Netgen (schematic vs extracted layout)
STA → OpenSTA (setup/hold at 50MHz, worst-case corners)

Verification sign-off criteria: DRC clean, LVS clean, setup/hold met at target frequency, all test vectors match golden model output within expected quantization tolerance.

9. MPW Tapeout Flow

RTL freeze and final functional sign-off
OpenLane hardening — synthesis through routing
DRC/LVS/STA sign-off
GDSII export and Caravel harness integration
Efabless precheck tool pass
Submit to Efabless MPW shuttle (Spring 2027)

Target shuttle: Efabless / SkyWater 130nm. Backup: GlobalFoundries 180nm via IHP if Sky130 shuttle timing doesn't align.

10. Timeline
Architecture finalization + module specs: May 2026
RTL development: September–November 2026
Integration + functional verificatio: December 2026
Synthesis + timing closure: January 2027 (OpenLane hardening, fix any synthesis issues)
Physical design (P&R): February 2027 (OpenROAD flow, floorplan through routing)
DRC/LVS/STA sign-off: March 2027 (Iterate until clean)
Tapeout submission: April–May 2027 (Buffer month before Spring 2027 shuttle)

11. Deliverables

RTL source (SystemVerilog, fully parameterized)
Gate-level netlist
GDSII layout
Verification reports (DRC, LVS, STA)
Python golden model
Final documentation and presentation